

Learning to Rank & Personalized Query Suggestion Tutorial Guide

NI-ADM — Advanced Data Mining
Czech Technical University in Prague

Task and feedback by Daniel Bohunek (Recombee)

Contents

Introduction and Research Questions	3
Part 1: Learning to Rank	4
1.1 Problem Formulation	4
1.2 Ranking Objectives	4
1.2.1 Pointwise Objective	4
1.2.2 Pairwise Objective	4
1.2.3 Listwise Objective: LambdaMART	5
1.3 Evaluation Metrics	5
1.3.1 NDCG (Normalised Discounted Cumulative Gain)	5
1.3.2 MRR (Mean Reciprocal Rank)	5
1.3.3 Recall@K	5
1.4 Position Bias and Debiasing	6
1.5 Feature Engineering for LTR	6
Part 2: Modern Approaches for Query Suggestion	7
2.1 Task Definition	7
2.2 Dataset: AOL4PS	7
2.3 Evaluation Protocol	7
2.3.1 Semantic NDCG@K	8
2.3.2 Semantic MRR	8
2.3.3 Hard Recall@K	8
2.4 Embedding Model	8
2.5 Baselines	8
2.5.1 Most Popular	8
2.5.2 Content-Based: Mean-Pooled User Profile	8
2.5.3 Collaborative Filtering: Truncated SVD	8
2.6 Two-Tower Model (Dual Encoder)	9
2.6.1 Architecture	9
2.6.2 Training with BPR Loss	9
2.6.3 Scalability at Inference	9
2.7 Sequential Model: GRU	9
2.7.1 Motivation	9
2.7.2 GRU (Gated Recurrent Unit)	10
2.7.3 Application to Query Suggestion	10
2.8 LoRA Fine-Tuning with Contrastive Loss	10
2.8.1 The Fine-Tuning Problem	10
2.8.2 Low-Rank Adaptation (LoRA)	10

2.8.3	Contrastive Loss (InfoNCE)	11
2.8.4	Trade-offs	11
2.9	Interpretation of Results	11
Part 3:	Student Assignment	13
3.1	Task	13
3.2	Constraints	13
3.3	Suggested Directions	13
3.4	Deliverables	13
	Glossary	14
	References	14

Introduction and Research Questions

This tutorial series investigates the problem of **personalised ranking**: given a user and a set of candidate items, produce an ordering from most relevant to least relevant. We study this problem in two concrete settings: (1) classical document ranking with hand-crafted features, and (2) personalised query suggestion using text embeddings and neural models.

The following research questions guide our experiments:

Research Question 1 (Pointwise vs. pairwise vs. listwise). How do pointwise, pairwise, and listwise learning-to-rank objectives compare in terms of $\text{NDCG}@K$ on the same feature set and model capacity? Does directly optimising the ranking metric (listwise) always outperform simpler surrogates?

Research Question 2 (Pretrained embeddings vs. learned representations). When candidate items carry textual descriptions, can a pretrained text embedding model (all-MiniLM-L6-v2) combined with a simple aggregation (mean pooling) outperform neural models trained from scratch on the target dataset? Under what data conditions does training on the target dataset start to pay off?

Research Question 3 (Collaborative vs. content-based signals). In an extremely sparse interaction setting ($>99.99\%$ sparsity), does collaborative filtering provide useful signal, or does it collapse to near-random performance? How does this compare to content-based methods that can generalise across semantically similar items?

Research Question 4 (Sequential vs. static user representations). Does modelling the temporal order of a user’s past interactions (via a GRU) improve next-item prediction compared to order-agnostic mean-pooling of the same interaction history?

Research Question 5 (Low-rank adaptation of embeddings). Can a lightweight low-rank adapter (LoRA), trained with contrastive loss on domain-specific user–query pairs, improve upon generic pretrained embeddings? What is the parameter–efficiency trade-off compared to full fine-tuning?

Research Question 6 (Latency–accuracy trade-off). What is the relationship between model complexity, inference latency, and ranking quality? Are there practical latency budgets below which only certain model families are viable?

Part 1: Learning to Rank

Accompanies `ltr_tutorial.ipynb` | Dataset: MQ2008 (LETOR 4.0)

1.1 Problem Formulation

Definition 1 (Learning to Rank). Given a query q and a set of candidate items $\mathcal{D}_q = \{d_1, \dots, d_n\}$, each described by a feature vector $\mathbf{x}_i = \phi(q, d_i) \in \mathbb{R}^m$, learn a scoring function $f(\mathbf{x})$ such that sorting items by $f(\mathbf{x}_i)$ in descending order produces a ranking that maximises a target metric (e.g. NDCG).

The key distinction from standard supervised learning: the loss is defined over *lists* (or pairs) of items within the same query, not over individual items in isolation. The relative ordering matters, not the absolute scores.

1.2 Ranking Objectives

1.2.1 Pointwise Objective

Reduce ranking to independent per-item prediction. For each (q, d_i) pair, predict the relevance label $y_i \in \{0, 1, 2\}$ using cross-entropy or MSE loss:

$$\mathcal{L}_{\text{pointwise}} = \sum_q \sum_{i \in \mathcal{D}_q} \ell(f(\mathbf{x}_i), y_i) \quad (1)$$

where ℓ is a classification (cross-entropy) or regression (squared error) loss.

At inference, items are sorted by $f(\mathbf{x}_i)$ or by expected relevance $\mathbb{E}[y_i | \mathbf{x}_i]$.

Limitation. The loss decomposes across items independently. The model has no mechanism to learn that errors at position 1 are more costly than errors at position 50, nor that the difficulty of distinguishing items varies by query.

1.2.2 Pairwise Objective

For each query, form ordered pairs (d_i, d_j) where $y_i > y_j$ and train a model to score d_i higher than d_j .

RankNet (Burgess et al., 2005) defines the probability that d_i should be ranked above d_j as:

$$P(d_i \succ d_j) = \sigma(f(\mathbf{x}_i) - f(\mathbf{x}_j)) \quad (2)$$

and minimises the cross-entropy between predicted and true ordering:

$$\mathcal{L}_{\text{RankNet}} = \sum_q \sum_{\substack{i, j \in \mathcal{D}_q \\ y_i > y_j}} -\log \sigma(f(\mathbf{x}_i) - f(\mathbf{x}_j)) \quad (3)$$

BPR (Bayesian Personalized Ranking; Rendle et al., 2009) uses the same functional form but applied to implicit feedback (clicks, not grades):

$$\mathcal{L}_{\text{BPR}} = \sum_{(u, i^+, i^-)} -\log \sigma(s(u, i^+) - s(u, i^-)) \quad (4)$$

where i^+ is an item the user interacted with and i^- is a randomly sampled negative.

Limitation. All pairs receive equal weight. Swapping positions 1 and 2 produces the same gradient as swapping positions 99 and 100, even though the former has a much larger impact on user experience.

1.2.3 Listwise Objective: LambdaMART

LambdaRank (Burges et al., 2006) modifies the pairwise gradient by weighting each pair by the *change in NDCG* that would result from swapping the two items:

$$\lambda_{ij} = \frac{\partial \mathcal{L}_{\text{RankNet}}}{\partial s_i} \cdot |\Delta \text{NDCG}_{ij}| \quad (5)$$

This “lambda trick” has a profound effect: pairs near the top of the ranking (where NDCG changes are large) receive much larger gradients than pairs deep in the list. The model focuses its capacity on getting the top positions right.

LambdaMART = LambdaRank + MART (gradient boosted decision trees). At each boosting step, a regression tree is fit to the lambda gradients defined in Equation 5. The LightGBM implementation uses `objective='lambdaRank'` and directly accepts the target NDCG cutoff as a hyperparameter.

Theoretical note. LambdaRank does *not* optimise any known loss function in closed form; the lambdas are defined procedurally. Nevertheless, empirical results consistently show that it outperforms methods that do optimise a well-defined surrogate. Donmez et al. (2009) showed that the lambda gradients can be interpreted as the gradient of a smooth upper bound on NDCG.

1.3 Evaluation Metrics

1.3.1 NDCG (Normalised Discounted Cumulative Gain)

For a ranked list of K items with relevance labels $\text{rel}_1, \dots, \text{rel}_K$:

$$\text{DCG@K} = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)} \quad (6)$$

$$\text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}} \quad (7)$$

where IDCG@K is the DCG of the ideal ranking (items sorted by decreasing relevance).

Properties: (1) bounded in $[0, 1]$; (2) position-sensitive via the logarithmic discount; (3) handles graded relevance (not just binary); (4) comparable across queries via normalisation.

1.3.2 MRR (Mean Reciprocal Rank)

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (8)$$

where rank_q is the position of the first relevant item for query q . $\text{MRR} = 0.5$ means the first relevant item is, on average, at position 2. MRR only considers the *first* relevant item; NDCG considers all relevant items in the top K .

1.3.3 Recall@K

$$\text{Recall@K} = \frac{|\{\text{relevant items in top-}K\}|}{|\{\text{all relevant items}\}|} \quad (9)$$

Measures coverage: what fraction of all relevant items did we retrieve? Useful in retrieval (stage 1) where we care about not missing good candidates.

1.4 Position Bias and Debiasing

In production systems, the training signal comes from **implicit feedback** (clicks), not editorial judgements. Clicks are biased by position: users examine and click items near the top of the list more often, regardless of true relevance.

Examination hypothesis (Richardson et al., 2007):

$$P(\text{click} \mid \text{item}, \text{position}) = P(\text{relevant} \mid \text{item}) \cdot P(\text{examined} \mid \text{position}) \quad (10)$$

The examination probability $P(\text{examined} \mid \text{position})$ decays with rank. Without correction, the model learns to reinforce whatever ranking was already shown (a *feedback loop*).

Debiasing approaches:

- **Position feature:** Include display position as a model feature during training; set it to a constant (e.g. 1) at serving time.
- **Inverse Propensity Scoring (IPS):** Weight each training example by $1/P(\text{examined} \mid \text{position})$, estimated from randomisation experiments or position-based models.
- **Dual Learning Algorithm (DLA):** Jointly learn a relevance model and a propensity model (Ai et al., 2018).

Caveat. Hager et al. (SIGIR 2024) showed on Baidu search logs that standard ULTR (Unbiased LTR) methods improved click prediction but did *not* consistently improve true ranking quality (NDCG). Simple debiasing (position as a feature) often suffices.

1.5 Feature Engineering for LTR

In tabular LTR (MQ2008, web search), feature engineering often matters **more than model choice**.

- **Query features:** query length, number of terms, whether it is navigational.
- **Document features:** PageRank, document length, freshness, authority score.
- **Query–document features:** BM25, TF-IDF cosine similarity, covered query term ratio, click-through rate for this (query, document) pair.
- **Context features:** time of day, device type, user location.
- **Cross-features:** Manually or automatically generated interactions (e.g. item category $\times$ user preferred category).

The MQ2008 dataset provides 46 pre-computed query–document features. A useful exercise is to train LambdaMART on all features, examine feature importance, then train on only the top 10 features and compare NDCG.

Part 2: Modern Approaches for Query Suggestion

Accompanies `query_suggestion_tutorial.ipynb` | Dataset: AOL4PS

2.1 Task Definition

Definition 2 (Next Query Prediction). Given a user u with a chronologically ordered search history $H_u = (q_1^u, q_2^u, \dots, q_T^u)$, and a candidate pool $\mathcal{C}$ of possible queries, produce a ranked list $\hat{R}_u \subset \mathcal{C}$ of K queries that maximises the probability of including the user’s actual next query q_{T+1}^u .

This task maps naturally to the recommendation framework:

Recommendation concept	Query suggestion
User	Searcher (anonymised ID)
Item	Query string
Interaction	User issued this query
Rating / relevance	Binary: searched (1) or not (0)

2.2 Dataset: AOL4PS

The AOL4PS dataset (Guo et al., 2021) is derived from AOL search logs (2006) and provides:

- **1,339,101 interactions** from **12,907 users**
- **429,286 unique query strings**
- **951,940 documents** with URLs and titles
- Pre-defined temporal split: `DataType` $\in \{0, 1, 2, 3\}$ mapping to history, train, validation, test
- Click position information (useful for position bias analysis)

Files are tab-separated; `QueryIndex` is a string identifier (e.g. q-0).

2.3 Evaluation Protocol

We adopt a strict temporal evaluation protocol:

1. **History + Train (DataType 0, 1):** Used to build user profiles, train models, and compute any derived features (embeddings, interaction matrices).
2. **Validation (DataType 2):** Used for hyperparameter tuning and model selection. All design decisions (learning rate, rank, architecture) must be justified by validation performance.
3. **Test (DataType 3):** Used *once* for final evaluation. No model parameter or hyperparameter may depend on test data.

Candidate pool. To make evaluation tractable, we restrict the candidate pool to the $N = 20,000$ most frequent queries from the training set.

Metrics. For each test user u :

1. Generate a ranked list $\hat{R}_u$ of $K = 10$ query suggestions.
2. Compare against the ground truth: the set of queries u actually searched in the test period.
3. Compute three metrics (defined formally below).

Report the **mean** of each metric across all test users. Additionally report **mean inference latency** (ms per user) to verify the 50 ms budget.

2.3.1 Semantic NDCG@K

Because many queries are semantically identical but syntactically different (“cat food” $\approx$ “food for cats”), we replace binary relevance with **soft semantic relevance**:

$$\text{rel}(q_{\text{pred}}, \mathcal{G}_u) = \max_{g \in \mathcal{G}_u} \cos(\mathbf{e}(q_{\text{pred}}), \mathbf{e}(g)) \cdot \mathbf{1}[\max \geq \tau] \quad (11)$$

where $\mathbf{e}(\cdot)$ is the sentence embedding function, $\mathcal{G}_u$ is the set of ground-truth queries for user u , and $\tau = 0.5$ is a relevance threshold.

This soft relevance is plugged into the standard DCG/NDCG formula (Section 1.3). We call the result **Semantic NDCG@K**.

2.3.2 Semantic MRR

The position of the first prediction with $\text{rel} > 0$ determines the reciprocal rank. Averaged across users yields Semantic MRR.

2.3.3 Hard Recall@K

Standard Recall@K with exact string matching. Included as a lower bound; Semantic NDCG is the primary metric.

2.4 Embedding Model

All methods use **all-MiniLM-L6-v2** (Wang et al., 2020; reimplemented in sentence-transformers) to embed query strings into $\mathbb{R}^{384}$. This model was trained on over 1 billion text pairs using a contrastive objective and produces high-quality sentence embeddings.

Embeddings are L_2 -normalised, so cosine similarity reduces to the dot product: $\cos(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$ when $\|\mathbf{a}\| = \|\mathbf{b}\| = 1$.

For queries not in the candidate pool (e.g. ground-truth queries, test pairs), embeddings are computed on-the-fly using the same model.

2.5 Baselines

2.5.1 Most Popular

Recommend the K most globally frequent queries that the user has not yet searched. Non-personalised. This baseline is included to measure whether personalisation adds value at all.

2.5.2 Content-Based: Mean-Pooled User Profile

1. For each user u , compute the mean embedding of their historical queries: $\mathbf{p}_u = \frac{1}{|H_u|} \sum_{q \in H_u} \mathbf{e}(q)$, then L_2 -normalise.
2. Score each candidate c as $s(u, c) = \mathbf{p}_u^\top \mathbf{e}(c)$.
3. Return the top- K candidates not in H_u .

This method has **zero trainable parameters**. It relies entirely on the quality of the pretrained embeddings. It addresses RQ 2.

2.5.3 Collaborative Filtering: Truncated SVD

Construct the user–query interaction matrix $\mathbf{M} \in \{0, 1\}^{|U| \times |C|}$ and compute a rank- k SVD approximation: $\mathbf{M} \approx \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.

The predicted affinity of user u for query c is $\hat{s}_{uc} = \mathbf{u}_u^\top \mathbf{v}_c$ where $\mathbf{u}_u$ and $\mathbf{v}_c$ are the corresponding rows of $\mathbf{U}\mathbf{\Sigma}^{1/2}$ and $\mathbf{V}\mathbf{\Sigma}^{1/2}$.

Expected failure mode. With $|\mathcal{C}| = 429,286$ unique queries and a median of ~ 60 queries per user, the matrix density is $\sim 0.001\%$. SVD captures the dominant singular vectors, which correspond to globally popular queries — effectively degenerating to a popularity baseline with slight user variation. This addresses RQ 3.

2.6 Two-Tower Model (Dual Encoder)

2.6.1 Architecture

Two separate feed-forward networks (“towers”) map inputs into a shared embedding space of dimension d :

$$\mathbf{u} = \text{norm}(f_{\theta_u}(\bar{\mathbf{e}}_u)) \in \mathbb{R}^d \quad (\text{user tower}) \quad (12)$$

$$\mathbf{q} = \text{norm}(f_{\theta_q}(\mathbf{e}_c)) \in \mathbb{R}^d \quad (\text{query tower}) \quad (13)$$

where $\bar{\mathbf{e}}_u = \text{mean}(\{\mathbf{e}(q) : q \in H_u\})$ is the mean-pooled user history and $\mathbf{e}_c$ is the pretrained embedding of candidate query c .

Each tower consists of: $\text{Linear}(d_{\text{in}}, 128) \rightarrow \text{ReLU} \rightarrow \text{LayerNorm} \rightarrow \text{Linear}(128, d) \rightarrow L_2\text{-norm}$.

The relevance score is the dot product: $s(u, c) = \mathbf{u}^\top \mathbf{q}$.

2.6.2 Training with BPR Loss

We train with the BPR objective (Eq. 4). For each user, we sample one positive (the last query in history within the candidate pool) and $N_{\text{neg}} = 4$ random negatives per training step.

Why BPR and not cross-entropy? BPR directly optimises the pairwise ordering, which aligns with the ranking task. Cross-entropy on binary labels (click/no-click) would be a pointwise objective, which as we saw in Section 1.2 is suboptimal for ranking.

2.6.3 Scalability at Inference

At serving time, all query-tower outputs $\mathbf{q}_c$ are **precomputed** and stored. For a new user, inference requires:

1. One mean-pooling of history embeddings: $O(|H_u| \cdot d_{\text{in}})$
2. One forward pass through the user tower: $O(d_{\text{in}} \cdot d)$
3. One matrix–vector product to score all candidates: $O(|\mathcal{C}| \cdot d)$
4. One partial sort for top- K : $O(|\mathcal{C}| \cdot \log K)$

With $|\mathcal{C}| = 20,000$ and $d = 64$, step 3 takes < 1 ms. For million-scale catalogs, an ANN index (e.g. FAISS) replaces the brute-force dot product.

2.7 Sequential Model: GRU

2.7.1 Motivation

Mean-pooling discards temporal order. If a user recently shifted from searching “travel” topics to “cooking” topics, the mean embedding sits somewhere between the two — which may not match either. A sequential model can learn to *attend more to recent queries*. This addresses RQ 4.

2.7.2 GRU (Gated Recurrent Unit)

A GRU (Cho et al., 2014) processes a sequence $(\mathbf{e}_1, \dots, \mathbf{e}_T)$ and produces a hidden state $\mathbf{h}_t$ at each step:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{e}_t]) \quad (\text{update gate}) \quad (14)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{e}_t]) \quad (\text{reset gate}) \quad (15)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{e}_t]) \quad (\text{candidate state}) \quad (16)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{new state}) \quad (17)$$

Intuition. The *update gate* $\mathbf{z}_t$ controls how much of the old memory to keep vs. replace. The *reset gate* $\mathbf{r}_t$ controls how much of the old memory to use when computing the new candidate. When $\mathbf{z}_t \approx 0$, the state is carried forward unchanged (“remember”); when $\mathbf{z}_t \approx 1$, it is replaced (“forget and update”).

2.7.3 Application to Query Suggestion

We use the final hidden state $\mathbf{h}_T$ as the user representation, project it to d dimensions, and L_2 -normalise. Candidate queries are projected through a separate linear layer (analogous to the query tower). Training uses BPR loss.

The architecture follows **GRU4Rec** (Hidasi et al., 2016) for session-based recommendation. A more recent alternative is **SASRec** (Kang & McAuley, 2018), which replaces the GRU with multi-head self-attention, allowing the model to attend to *any* position in the history, not just propagating information sequentially.

2.8 LoRA Fine-Tuning with Contrastive Loss

This section addresses RQ 5.

2.8.1 The Fine-Tuning Problem

The pretrained embedding model $\mathbf{e}(\cdot)$ maps texts into a generic semantic space. For our specific domain (search queries + user behaviour), we might benefit from an adapted embedding space where user profiles are closer to their future queries.

Full fine-tuning would update all parameters of the embedding model ($\sim 22\text{M}$ parameters for all-MiniLM-L6-v2). This is:

- Computationally expensive (GPU hours)
- Prone to overfitting on small datasets
- Requires recomputing all embeddings after each update

2.8.2 Low-Rank Adaptation (LoRA)

LoRA (Hu et al., 2021) offers a parameter-efficient alternative. Instead of modifying the full weight matrix $\mathbf{W} \in \mathbb{R}^{d \times d}$, add a low-rank perturbation:

$$\mathbf{W}' = \mathbf{W} + \mathbf{A}\mathbf{B} \quad (18)$$

where $\mathbf{A} \in \mathbb{R}^{d \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times d}$ with rank $r \ll d$.

Parameter count comparison (for $d = 384$, $r = 16$):

Method	Trainable params	Ratio
Full matrix	$384^2 = 147,456$	100%
LoRA (rank 16)	$2 \times 384 \times 16 = 12,288$	8.3%
LoRA (rank 4)	$2 \times 384 \times 4 = 3,072$	2.1%

Initialisation. $\mathbf{A}$ is initialised with small random values; $\mathbf{B}$ is initialised to zero. Therefore $\mathbf{AB} = \mathbf{0}$ at initialisation and the model starts from the exact pretrained weights. Training gradually learns a task-specific correction.

In our implementation, we apply separate LoRA adapters to the user and query representations (a “LoRA Two-Tower”).

2.8.3 Contrastive Loss (InfoNCE)

The LoRA model is trained with **InfoNCE** loss (van den Oord et al., 2018), also known as NT-Xent (Chen et al., 2020). This is the same loss family used in CLIP (Radford et al., 2021) and SimCLR.

Given a user representation $\mathbf{u}$, a positive query $\mathbf{q}^+$, and N negative queries $\mathbf{q}_1^-, \dots, \mathbf{q}_N^-$:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\mathbf{u}^\top \mathbf{q}^+ / \tau)}{\exp(\mathbf{u}^\top \mathbf{q}^+ / \tau) + \sum_{j=1}^N \exp(\mathbf{u}^\top \mathbf{q}_j^- / \tau)} \quad (19)$$

This is equivalent to $(N + 1)$ -way cross-entropy classification: identify the positive among $N + 1$ candidates.

Temperature τ . Controls the sharpness of the softmax distribution:

- $\tau \rightarrow 0$: the loss approaches a hard argmax — only the single highest-scoring negative matters.
- $\tau \rightarrow \infty$: all candidates contribute equally — the loss becomes uninformative.
- Typical values: $\tau \in [0.05, 0.2]$. We use $\tau = 0.1$.

BPR vs. InfoNCE. BPR processes one negative at a time; InfoNCE considers all negatives jointly in a softmax. InfoNCE has been shown to provide more informative gradients when multiple negatives are available, as it implicitly performs hard negative mining through the softmax normalisation (Robinson et al., 2021).

2.8.4 Trade-offs

- + Few trainable parameters $\rightarrow$ fast training, less overfitting.
- + Starts from high-quality pretrained embeddings $\rightarrow$ good initialisation.
- After retraining the adapter, **all candidate embeddings must be recomputed** (the query adapter $\mathbf{B}_q$ changed). For 20K candidates this takes seconds; for millions it may take hours.
- In practice, LoRA adapters are trained once when sufficient data accumulates and deployed for weeks/months until the next retraining cycle.

2.9 Interpretation of Results

On the AOL4PS dataset, the expected ranking of methods is:

Method	Expected behaviour and explanation
Content-Based	Strongest baseline. Leverages excellent pretrained embeddings with zero training.
LoRA	Close second. The domain adaptation slightly reshapes the embedding space but has limited training data.
GRU	Moderate. Captures temporal patterns but limited by small dataset and short histories.
Two-Tower	Moderate. Learns a transformation but with only 12K training samples the towers may not converge well.
Most Popular	Weak. Non-personalised; only works for users whose interests align with the global distribution.
CF (SVD)	Weakest. Interaction matrix too sparse; treats queries as opaque IDs.

Key insight (RQ 2). When the pretrained representation is strong and training data is limited, *not training at all* (Content-Based) can outperform models that attempt to learn from the data. This is analogous to the bias–variance trade-off: the learned models have lower bias but higher variance, and with insufficient data the variance dominates.

This result is dataset-dependent. With millions of users and interactions (e.g. Google, Spotify), trained models consistently outperform simple baselines.

Part 3: Student Assignment

Accompanies `assignment.ipynb`

3.1 Task

Design and implement a method that achieves a higher **Semantic NDCG@10** than the tutorial baselines on the AOL4PS test set (DataType = 3).

3.2 Constraints

1. **Latency budget:** The `recommend(user_id, n)` function must execute in <50 ms per user on average (single CPU core). This simulates a production constraint where query suggestions must appear in real time.
2. **No test leakage:** Test ground truth may not be used for training, tuning, or feature engineering. Use the validation set for all design decisions.
3. **Reproducibility:** Set random seeds and document your hyperparameters.

3.3 Suggested Directions

- **Exponential decay weighting.** Replace the uniform mean with $\mathbf{p}_u = \sum_t \lambda^{T-t} \mathbf{e}(q_t^u)$ (normalised), where $\lambda \in (0, 1)$ is a decay factor tuned on validation. This gives more weight to recent queries.
- **Hard negative sampling.** Replace random negatives with “hard” negatives: queries that are semantically close to the positive but were not searched. This forces the model to learn finer distinctions.
- **Hybrid score fusion.** Combine scores from multiple retrievers: $s_{\text{final}} = \alpha_1 s_{\text{content}} + \alpha_2 s_{\text{CF}} + \alpha_3 s_{\text{popularity}} + \dots$. Tune α_i on validation via grid search or logistic regression.
- **Self-attention (SASRec).** Replace the GRU with a causal Transformer encoder over the query sequence. Self-attention can attend to any position, potentially capturing long-range dependencies better.
- **Session-aware modelling.** Segment user history into sessions (e.g. by time gaps >30 minutes). Model intra-session intent separately from long-term interests.
- **Query co-occurrence graph.** Build a graph where an edge (q_i, q_j) exists if the same user searched both queries in the same session. Apply node2vec or a GNN to learn query representations that capture co-occurrence structure.
- **Two-stage pipeline.** Stage 1: fast retrieval (ANN or content-based) to get 100 candidates. Stage 2: a richer model (LambdaMART with features from multiple retrievers) re-ranks to the final 10.

3.4 Deliverables

1. Completed `assignment.ipynb` with your implementation and evaluation.
2. Brief write-up (in the notebook): describe your method, report validation and test metrics, discuss what worked and what did not.
3. Ablation: if you combine multiple ideas, show the contribution of each.

Glossary

ANN	Approximate Nearest Neighbour. A data structure and algorithm for finding the k closest vectors to a query vector in sub-linear time. Examples: FAISS (Meta), Annoy (Spotify), HNSW.
BPR	Bayesian Personalized Ranking (Rendle et al., 2009). A pairwise loss for implicit-feedback recommendation: $\mathcal{L} = -\log \sigma(s^+ - s^-)$.
Cold-start	The difficulty of making predictions for new users (no history) or new items (no interactions). Content-based methods alleviate item cold-start; popularity alleviates user cold-start.
Contrastive learning	A training paradigm where the model learns to pull similar examples together and push dissimilar examples apart in embedding space. InfoNCE (Eq. 19) is the standard loss.
Cosine similarity	$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\ \mathbf{a}\ \ \mathbf{b}\ }$. For L_2 -normalised vectors, $\cos = \mathbf{a}^\top \mathbf{b}$ (dot product).
Dot product	$\mathbf{a}^\top \mathbf{b} = \sum_i a_i b_i$.
Embedding	A dense vector representation of a discrete object (word, query, user). Similar objects have similar embeddings (high cosine similarity).
GRU	Gated Recurrent Unit (Cho et al., 2014). An RNN variant with update and reset gates that control information flow through the sequence.
Implicit feedback	User signals that indicate preference indirectly: clicks, searches, purchases, dwell time. Contrast with explicit ratings.
InfoNCE	$(N+1)$ -way cross-entropy contrastive loss (van den Oord et al., 2018).
LambdaMART	Gradient boosted trees with lambda gradients that weight pairwise losses by $ \Delta \text{NDCG} $ (Burges et al., 2006).
LoRA	Low-Rank Adaptation (Hu et al., 2021). Adds trainable AB to frozen weights, reducing trainable parameters by 10–100×.
NDCG	Normalised Discounted Cumulative Gain. $\text{NDCG}@K = \text{DCG}@K / \text{IDCG}@K$.
Negative sampling	Selecting items the user did not interact with as negative training examples. Random sampling is simplest; hard negative sampling selects challenging examples near the decision boundary.
Two-Tower	A neural architecture with separate user and item encoders. Item encodings are precomputed, enabling fast ANN retrieval at serving time.

References

- [1] Burges, C. et al. (2005). Learning to rank using gradient descent. *ICML*.
- [2] Burges, C. et al. (2006). Learning to rank with nonsmooth cost functions. *NIPS*.
- [3] Rendle, S. et al. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. *UAI*.
- [4] Cho, K. et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder. *EMNLP*.
- [5] Hidasi, B. et al. (2016). Session-based Recommendations with Recurrent Neural Networks. *ICLR*.

- [6] Kang, W. & McAuley, J. (2018). Self-Attentive Sequential Recommendation. *ICDM*.
- [7] van den Oord, A. et al. (2018). Representation Learning with Contrastive Predictive Coding. *arXiv:1807.03748*.
- [8] Chen, T. et al. (2020). A Simple Framework for Contrastive Learning of Visual Representations (SimCLR). *ICML*.
- [9] Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP). *ICML*.
- [10] Hu, E. et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022*.
- [11] Guo, Q. et al. (2021). AOL4PS: A Large-scale Data Set for Personalized Search. *Data Intelligence*.
- [12] Hager, L. et al. (2024). Unbiased Learning to Rank Meets Reality. *SIGIR*.
- [13] Robinson, J. et al. (2021). Contrastive Learning with Hard Negative Samples. *ICLR*.